1. You **ignore the lang parameter** from the request. You always pass `"hun"` to Tesseract. If the user sends `"ENG"`, it won't be respected.
2. You return the **full Tesseract response object**, not just the text.
   - `Tesseract.recognize` returns `{ data: { text, words, … } }`.
   - Your `recognizeData(parsedText, lang)` probably expects a **string**, not the full object.

3️⃣ InvoiceController usage

```js
const parsedText = await extractTextFromDocument(fileName);
const data = await recognizeData(parsedText, lang);
```

- `parsedText` is the full object from Tesseract, not just text.
- `recognizeData` expects a **string** to interpolate into your prompt:

```js
const prompt = buildPrompt("recognize.deadline.HUN", { invoice: parsedText });
```

- This will inject `[object Object]` into your prompt if you don't extract `text`.

4️⃣ Corrected TesseractService

```js
import Tesseract from "tesseract.js";

export async function extractTextFromDocument(docPath, lang = "eng") {
    try {
        const { data: { text } } = await Tesseract.recognize(docPath, lang.toLowerCa
        return text; // only return the recognized text
    } catch (error) {
        throw new Error("OCR failed: " + error.message);
    }
}
```

- Uses `lang` from the request and converts to lowercase (Tesseract expects `"eng"` or `"hun"`).
- Returns only `text` for Ollama.

5️⃣ Corrected InvoiceController

```js
export const parseAndRecognizeInvoicesDate = async (req, res) => {
    try {
        const { fileName, lang } = req.body;

        // Make sure to include the correct path
        const path = `./uploads/${fileName}`; // or wherever your test files are
        const parsedText = await extractTextFromDocument(path, lang);

        const data = await recognizeData(parsedText, lang);
        res.json(data);
    } catch (error) {
        console.error(error);
        res.status(500).json({ error: "Failed to parse and recognize data" });
    }
};
```

- Now `parsedText` is a **string**, ready for `recognizeData`.
- `path` should point to the **actual file location** on your server.

6️⃣ Summary of likely causes of failure

1. Passing `fileName` instead of a **full path** → Tesseract cannot find the file.
2. Hardcoded `"hun"` → ignores `lang` parameter.
3. Returning the **full Tesseract object** → `[object Object]` gets passed to Ollama instead of plain text.

I can also write a **full working example with uploads** so you don't have to manually provide `fileName` and the endpoint will accept files directly and process OCR → LLM.

Do you want me to do that?

> Okay now I have one workflow which can parse an invoice and send to the LLM to recognize the payment date of the invoice. How to go further? E.g. I want also to recognize the amount and the issue date.

Perfect! Now that you have **one workflow working**, you can extend it in a **structured, scalable way** rather than just copying code. The key is to make the **workflow modular and prompt-driven**. Here's how to go further:

1️⃣ Use your prompt JSON for multiple data points

You already have something like this:

```json
"recognize": {
  "deadline": {
    "HUN": "...",
    "ENG": "..."
  }
}
```

You can extend it:

```json
"recognize": {
  "deadline": {
    "HUN": "...",
    "ENG": "..."
  },
  "amount": {
    "HUN": "Csatolom a számla beolvasott szövegét, mondd meg a fizetendő összeget. S
    "ENG": "I am attaching the invoice text, please tell me the total amount. Text:
  },
  "issueDate": {
    "HUN": "Csatolom a számla beolvasott szövegét, mondd meg a kiállítás dátumát. Sz
    "ENG": "I am attaching the invoice text, please tell me the issue date. Text: {{
  }
}
```

✅ This allows your service to **reuse the same buildPrompt function** with different keys.

2️⃣ Make a generic recognizeField method

Instead of having separate methods for deadline, amount, issue date:

```js
// services/ollamaService.js
import ollama from "ollama";
import { buildPrompt } from "../utils/promptLoader.js";

export async function recognizeField(parsedText, field, lang = "HUN") {
  const promptName = `recognize.${field}.${lang}`;
  const prompt = buildPrompt(promptName, { invoice: parsedText });

  const response = await ollama.generate({
    model: "llama3.1:8b",
    prompt
  });

  return response.text; // only the text output
}
```

- `field` can be `"deadline"`, `"amount"`, `"issueDate"`, etc.
- `parsedText` is always the string from Tesseract.

3️⃣ Update your workflow controller

controllers/invoiceController.js

```js
import * as tesseractService from "../services/tesseractService.js";
import * as ollamaService from "../services/ollamaService.js";

export const parseAndRecognizeInvoice = async (req, res) => {
  try {
    const { fileName, lang } = req.body;
    const path = `./uploads/${fileName}`;

    // 1️⃣ OCR
    const parsedText = await tesseractService.extractTextFromDocument(path, lang);

    // 2️⃣ Recognize multiple fields
    const deadline = await ollamaService.recognizeField(parsedText, "deadline", lang
    const amount = await ollamaService.recognizeField(parsedText, "amount", lang);
    const issueDate = await ollamaService.recognizeField(parsedText, "issueDate", la

    res.json({ parsedText, deadline, amount, issueDate });
  } catch (err) {
    console.error(err);
    res.status(500).json({ error: "Failed to parse and recognize invoice" });
  }
};
```

4️⃣ Optional: parallel requests for efficiency

If you have **many fields**, you can run them in parallel instead of sequentially:

```js
const [deadline, amount, issueDate] = await Promise.all([
  ollamaService.recognizeField(parsedText, "deadline", lang),
  ollamaService.recognizeField(parsedText, "amount", lang),
  ollamaService.recognizeField(parsedText, "issueDate", lang)
]);
```

This speeds up LLM calls.

5️⃣ Benefits of this approach

1. **Reusable** → same function works for any field by adding to your JSON prompts.
2. **Extensible** → later add `"supplier"`, `"invoiceNumber"`, `"tax"` without touching service logic.
3. **Clean structure** → TesseractService only parses, OllamaService only interprets, controller orchestrates workflow.

If you want, I can also suggest a **next-level improvement**: instead of calling the LLM multiple times for each field, you can **ask the LLM to extract all required fields in one prompt**. This reduces API calls and can give more consistent results.

Do you want me to show that approach?

> I rewrite my recognizeData in ollamaService:

```
export async function recognizeData(parsedText, field, lang = "ENG") {
  const promptName = recognize.${field}.${lang};
  const prompt = buildPrompt(promptName, {invoice: parsedText});

  return await ollama.generate({
    model: "llama3.1:8b",
    prompt
  });
}
```

> And the invoiceController:

```
import { extractTextFromDocument } from "../services/tesseractService.js";
import { recognizeData } from "../services/ollamaService.js";

export const parseAndRecognizeInvoicesData = async (req, res) => {
  try {
    const { fileName, lang } = req.body;

    const path = ./documents/${fileName};
    const parsedText = await extractTextFromDocument(path, lang);
    const deadline = await recognizeData(parsedText, "deadline", lang);
    const amount = await recognizeData(parsedText, "amount", lang);
    const issued = await recognizeData(parsedText, "issued", lang);

    res.json({deadline, amount, issued});
  } catch (error) {
    console.error(error);
    res.status(500).json({ error: "Failed to parse and recognize data" });
  }
};
```

> This is invoice routes:

```
import express from "express";
import { parseAndRecognizeInvoicesData } from "../controllers/invoiceController.js";

const invoiceRouter = express.Router();

invoiceRouter.post("/process", parseAndRecognizeInvoicesData);
```

> **Show full message**

Ah, I see exactly what's happening. The `ollama.generate()` response is **not just plain text**, it usually includes some metadata like a `text` key, sometimes `context`, depending on the version. Right now, your controller returns the full object from `ollama.generate()`, so you see extra keys.

1️⃣ Modify recognizeData to return only the text

```js
export async function recognizeData(parsedText, field, lang = "ENG") {
    const promptName = `recognize.${field}.${lang}`;
    const prompt = buildPrompt(promptName, { invoice: parsedText });

    const response = await ollama.generate({
        model: "llama3.1:8b",
        prompt
    });

    // return only the main text output
    return response.text;
}
```

✅ Now, `deadline`, `amount`, `issued` in your controller will just be strings, not objects with extra keys.

2️⃣ Optional: trim whitespace

LLMs sometimes return text with newlines or extra spaces:

```js
return response.text.trim();
```

This ensures clean values in your JSON response.

3️⃣ Updated controller

```js
export const parseAndRecognizeInvoicesData = async (req, res) => {
    try {
        const { fileName, lang } = req.body;

        const path = `./documents/${fileName}`;
        const parsedText = await extractTextFromDocument(path, lang);

        const deadline = await recognizeData(parsedText, "deadline", lang);
        const amount = await recognizeData(parsedText, "amount", lang);
        const issued = await recognizeData(parsedText, "issued", lang);

        res.json({ deadline, amount, issued }); // now all plain strings
    } catch (error) {
        console.error(error);
        res.status(500).json({ error: "Failed to parse and recognize data" });
    }
};
```

✅ After this change, your API response will look like:

```json
{
  "deadline": "2025-10-30",
  "amount": "10.0 USD",
  "issued": "2025-09-26"
}
```

No extra `context` or metadata included.

If you want, I can also show a **more efficient version** where you call LLM **once** to extract all fields at the same time instead of three separate calls. This reduces latency and may improve consistency.

Do you want me to do that?

> Okay currently I have a folder with the invoices on my backend because for now I don't have a frontend but later I want the user to be able to upload documents on the frontend and use those documents to parse and recognize. How could it be implemented?

Perfect question 👌 You're basically describing the **file upload flow** from a React frontend → Express backend → OCR (Tesseract) → LLM (Ollama).

Right now you hardcode invoices in a backend `documents/` folder. Later you want:

1. User uploads an invoice (image/PDF) from the frontend.
2. Backend saves it in your `documents/` folder (or cloud storage later).
3. Backend calls Tesseract to parse it.
4. Backend calls Ollama to recognize structured data.
5. Backend sends structured JSON back to frontend.

🔷 Step 1: Backend – Setup File Upload

Use **multer** (Express middleware for handling `multipart/form-data`).

Install:

```bash
npm install multer
```

Create `uploadMiddleware.js` in `middlewares/`:

```js
import multer from "multer";
import path from "path";

// Configure storage
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, "./documents"); // save in backend's documents folder
  },
  filename: (req, file, cb) => {
    const uniqueSuffix = Date.now() + "-" + Math.round(Math.random() * 1e9);
    cb(null, uniqueSuffix + path.extname(file.originalname));
  },
});

export const upload = multer({ storage });
```

🔷 Step 2: Backend – Update Route

In `invoiceRoutes.js`:

```js
import express from "express";
import { parseAndRecognizeInvoicesData } from "../controllers/invoiceController.js";
import { upload } from "../middlewares/uploadMiddleware.js";

const invoiceRouter = express.Router();

// upload single file with field name "invoice"
invoiceRouter.post("/upload", upload.single("invoice"), parseAndRecognizeInvoicesDat

export default invoiceRouter;
```

🔷 Step 3: Backend – Update Controller

In `invoiceController.js`, you'll now receive the uploaded file:

```js
import { extractTextFromDocument } from "../services/tesseractService.js";
import { recognizeData } from "../services/ollamaService.js";

export const parseAndRecognizeInvoicesData = async (req, res) => {
    try {
        const lang = req.body.lang || "ENG";  // default to ENG
        const filePath = req.file.path; // multer saves file path

        const parsedText = await extractTextFromDocument(filePath, lang);

        const deadline = await recognizeData(parsedText, "deadline", lang);
        const amount = await recognizeData(parsedText, "amount", lang);
        const issued = await recognizeData(parsedText, "issued", lang);

        res.json({ deadline, amount, issued });
    } catch (error) {
        console.error(error);
        res.status(500).json({ error: "Failed to parse and recognize data" });
    }
};
```

🔷 Step 4: Frontend – React Upload Form

Later in your React app, you'll do something like:

```jsx
import { useState } from "react";

function InvoiceUploader() {
  const [file, setFile] = useState(null);
  const [result, setResult] = useState(null);

  const handleSubmit = async (e) => {
    e.preventDefault();
    const formData = new FormData();
    formData.append("invoice", file);
    formData.append("lang", "HUN"); // or ENG

    const res = await fetch("http://localhost:5000/invoices/upload", {
      method: "POST",
      body: formData,
    });

    const data = await res.json();
    setResult(data);
  };

  return (
    <div>
      <form onSubmit={handleSubmit}>
        <input
          type="file"
          onChange={(e) => setFile(e.target.files[0])}
          required
        />
        <button type="submit">Upload</button>
      </form>

      {result && (
        <pre>{JSON.stringify(result, null, 2)}</pre>
      )}
    </div>
  );
}

export default InvoiceUploader;
```

✅ Workflow now looks like this:

Frontend uploads → Express saves file → Tesseract extracts text → Ollama recognizes structured data → JSON response goes back to frontend.

Would you like me to show you how to **merge all fields into a single Ollama call** at this point (instead of 3 separate ones)? It will make this API a lot faster and more consistent.

> Please explain multer and the updated route and controller in details

Great! Let's go step by step and break down **multer**, the updated **route**, and the updated **controller**, so you clearly see what's happening in each part.

🔷 What is Multer?

- Multer is a middleware for **Express.js** that helps you handle `multipart/form-data` – this is the encoding browsers use when you upload files with `<input type="file">`.
- Without it, Express can only parse `application/json` or `application/x-www-form-urlencoded`, but **file uploads won't work**.
- Multer:
  - Reads the incoming file stream from the request.
  - Saves the file somewhere on your server (in memory or disk).
  - Puts information about the file into `req.file` (if single upload) or `req.files` (if multiple).

🔷 Middleware Setup (uploadMiddleware.js)

```js
import multer from "multer";
import path from "path";

// Configure storage
const storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, "./documents"); // where to save uploaded files
  },
  filename: (req, file, cb) => {
    const uniqueSuffix = Date.now() + "-" + Math.round(Math.random() * 1e9);
    cb(null, uniqueSuffix + path.extname(file.originalname));
  },
});

// Export multer with the config
export const upload = multer({ storage });
```

What happens here:

1. **multer.diskStorage()**: Tells multer to save files on your disk instead of keeping them only in memory.
2. `destination`: A function that tells multer *where* to save the file. We use `./documents`, so invoices end up in your backend's `documents/` folder.
3. `filename`: A function that tells multer *how to name* the saved file.
   - We add a unique suffix with `Date.now()` + random number.
   - We preserve the original file extension using `path.extname(file.originalname)`.
   - Example: `1727446031012-834923877.png`.

So multer **saves the file** and attaches its info to `req.file`.

🔷 Updated Route (invoiceRoutes.js)

```js
import express from "express";
import { parseAndRecognizeInvoicesData } from "../controllers/invoiceController.js";
import { upload } from "../middlewares/uploadMiddleware.js";

const invoiceRouter = express.Router();

// POST /invoices/upload
invoiceRouter.post("/upload", upload.single("invoice"), parseAndRecognizeInvoicesDat

export default invoiceRouter;
```

What happens here:

1. The route is `/invoices/upload`.
2. `upload.single("invoice")`:
   - This tells multer to expect **one file** uploaded under the field name `invoice`.
   - Example from frontend:

```js
formData.append("invoice", file);
```

3. Once multer runs, it saves the file on disk and adds `req.file` with metadata:

```js
{
  fieldname: 'invoice',
  originalname: 'invoice.png',
  encoding: '7bit',
  mimetype: 'image/png',
  destination: './documents',
  filename: '1727446031012-834923877.png',
  path: 'documents/1727446031012-834923877.png',
  size: 123456
}
```

4. After multer finishes, Express calls `parseAndRecognizeInvoicesData`.

🔷 Updated Controller (invoiceController.js)

```js
import { extractTextFromDocument } from "../services/tesseractService.js";
import { recognizeData } from "../services/ollamaService.js";

export const parseAndRecognizeInvoicesData = async (req, res) => {
    try {
        const lang = req.body.lang || "ENG";  // user can send "HUN" or fallback to
        const filePath = req.file.path;       // multer gave us this path

        // 1. Extract text from the uploaded file with Tesseract
        const parsedText = await extractTextFromDocument(filePath, lang);

        // 2. Send extracted text to Ollama to recognize structured fields
        const deadline = await recognizeData(parsedText, "deadline", lang);
        const amount   = await recognizeData(parsedText, "amount", lang);
        const issued   = await recognizeData(parsedText, "issued", lang);

        // 3. Send structured result back to frontend
        res.json({ deadline, amount, issued });
    } catch (error) {
        console.error(error);
        res.status(500).json({ error: "Failed to parse and recognize data" });
    }
};
```

What happens here:

1. We read `req.body.lang` (e.g. `"HUN"`), default to `"ENG"`.
2. Multer already saved the file, and put its path in `req.file.path`.
Example: `documents/1727446031012-834923877.png`
3. We pass this file path into `extractTextFromDocument()`, which runs Tesseract OCR.
4. Tesseract returns the **parsed raw text** of the invoice.
5. We send this text into Ollama with different prompts (deadline, amount, issued).
6. Finally, we send the structured JSON response back to the frontend.

🔷 Summary of the Flow

- **Frontend** sends file + language → `FormData`.
- **Route** receives request → multer saves file → controller takes over.
- **Controller** uses `req.file.path` + `req.body.lang`.
- **Tesseract Service** extracts text.
- **Ollama Service** interprets the text into structured fields.
- **Controller** returns JSON to frontend.

👉 This way, each part is responsible for its own job:

- **multer**: file handling
- **controller**: orchestration (connects services together)
- **services**: actual business logic (OCR, LLM)

Do you want me to also show you how to **send multiple fields (deadline, amount, issued) in a single Ollama call**, instead of making 3 separate ones? This would be more efficient.